

# Exam

Modern Code Development  
ModernCode (MTD.ba VZ)  
SS252025

Surname(s): .....

Given name(s): .....

Student ID / Mat.-Nr.: .....

|               | maximum | achieved |
|---------------|---------|----------|
| Question 1    | 15      |          |
| Question 2    | 15      |          |
| Question 3    | 15      |          |
| Question 4    | 12      |          |
| Question 5    | 15      |          |
| Question 6    | 15      |          |
| Question 7    | 18      |          |
| <b>Total:</b> | 105     |          |

**Time allowed: 60 minutes**

**DO NOT TURN OVER UNTIL INSTRUCTED!**

**Question 1 (15 points)** Write a method `reverseArray` that takes an integer array and reverses it in-place (modifies the original array, doesn't create a new one). After writing the code, explain:

- Why this is an "in-place" algorithm (what does that mean?)
- How your algorithm works (the swapping strategy)
- Why you only need to iterate through half the array
- What would happen if you tried to swap elements beyond the midpoint

**Question 2 (15 points)** Trace through the execution of `factorial(4)` using a recursive implementation. Show:

- Each recursive call that is made (in order)
- The stack frames that are created (what variables/parameters are in each frame)
- When the base case is reached
- How values are returned and combined
- The final result

Draw a simple diagram or use a table to show the call sequence and stack growth. Explain why the stack "unwinds" after the base case is reached.

**Question 3 (15 points)** Sorting algorithms can be categorized by several properties. Explain each of these concepts and give examples:

- **Stable vs Unstable:** What does this mean? Give an example of a stable and an unstable sort.
- **In-place vs Out-of-place:** What is the memory difference? Which sorts are which?
- **Adaptive vs Non-adaptive:** What does adaptive mean? Which sorts benefit from partially sorted data?

Why do these properties matter when choosing a sorting algorithm?

**Question 4 (12 points)** Explain the difference between method parameters and return values as ways to pass data.

- When should you use parameters vs when should you use return values?
- What is the Command-Query Separation principle?
- Give an example of a method that violates this principle
- Show how to refactor it to follow the principle

**Question 5 (15 points)** You need to process a 2D grid to count neighbors (like in Minesweeper). For a cell at position `[r] [c]`, you need to check all 8 surrounding cells. Explain:

- Why you must check boundaries before accessing `grid[r-1][c-1]`
- What happens if you don't check boundaries (what error occurs?)
- Describe the pattern for checking all 8 neighbors safely
- What are the three types of cells in a grid (corner, edge, interior) and how many neighbors does each have?
- Why is a helper method like `isValid(grid, r, c)` useful?

**Question 6 (15 points)** Explain why Binary Search can find an item in a sorted array of 1,000,000 elements in at most 20 steps, while Linear Search might need 1,000,000 steps in the worst case. Your answer should include:

- How Binary Search eliminates half the search space each step
- Why this logarithmic behavior is so powerful
- What the trade-off is that makes Binary Search possible (what requirement must be met?)
- A simple calculation showing why 1,000,000 elements needs at most 20 steps
- When Linear Search might still be the better choice

**Question 7 (18 points)** Compare top-down and bottom-up design approaches for breaking down a problem into methods. Your answer should explain:

- How top-down design works (start with the big picture)
- How bottom-up design works (start with small pieces)
- When each approach is more appropriate
- How they complement each other in real-world development
- Give a concrete example (like a game turn or data processing task) showing both approaches